Практическая работа №2

Базовые CRUD-операции через консоль

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Реализовать консольное приложение «Список задач», выполняющее четыре базовые операции с данными (CRUD): добавление, просмотр, изменение статуса и удаление задач, с использованием параметризованных SQL-запросов.

Технологии: Python 3; модуль sqlite3; SQL: INSERT, SELECT, UPDATE, DELETE; параметризованные запросы с плейсхолдером ?.

Реализация: Каждая операция открывает соединение, выполняет запрос, при изменении данных вызывает commit() и закрывает соединение. Все значения передаются через плейсхолдер ? и кортеж, что защищает от SQL-инъекций. Ввод id проверяется методом isdigit(), а результат UPDATE/DELETE — через cursor.rowcount (0 — задача не найдена). Удаление требует подтверждения.

Основные модули:

- `src/init_db.py, src/check_db.py` — создание и проверка базы (из работы №1).
- `src/crud_console.py` — консольное приложение с меню: list_tasks() (READ), add_task() (CREATE), mark_done() (UPDATE), delete_task() (DELETE), main() — цикл меню.

Ход работы

Шаг 1. Подготовка

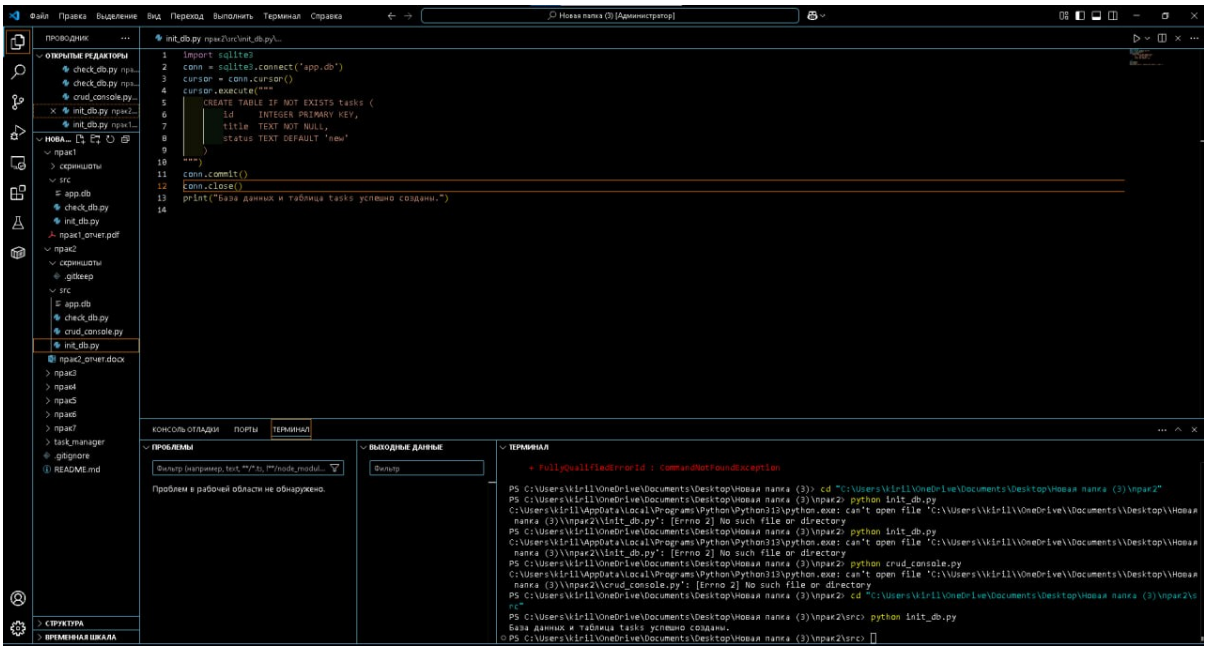


Рисунок 1 — структура проекта и запуск init_db.py в папке прак2/src

Шаги 2–7. Каркас приложения с меню и функции CRUD

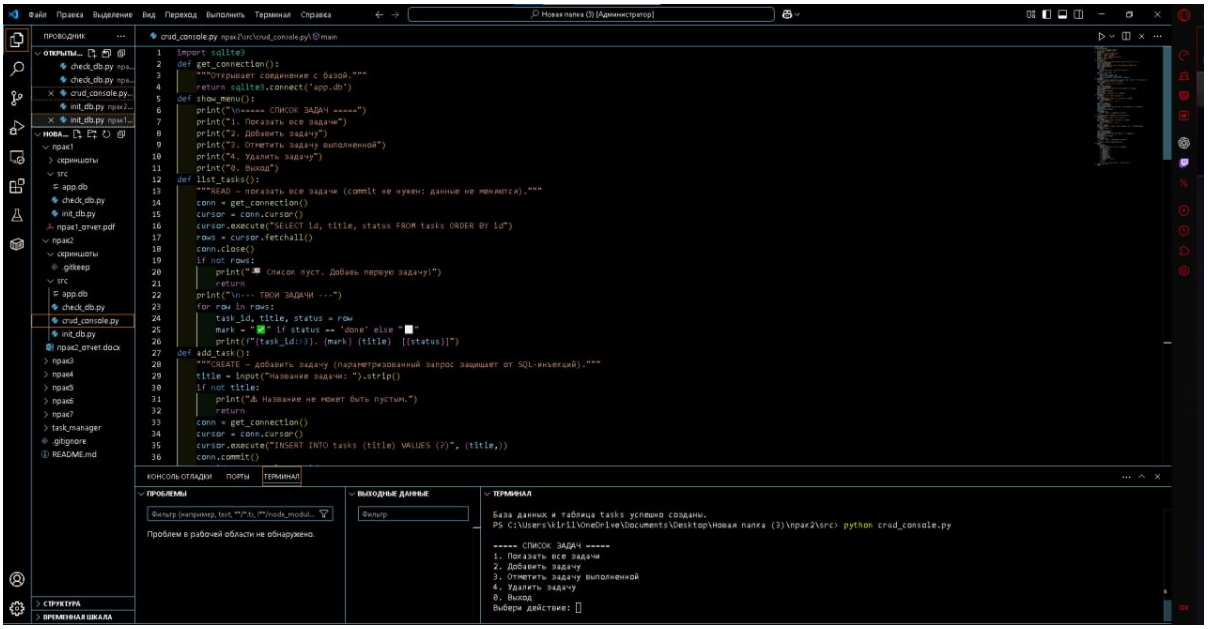


Рисунок 2 — код crud_console.py и меню программы

Шаг 8. Сценарий проверки

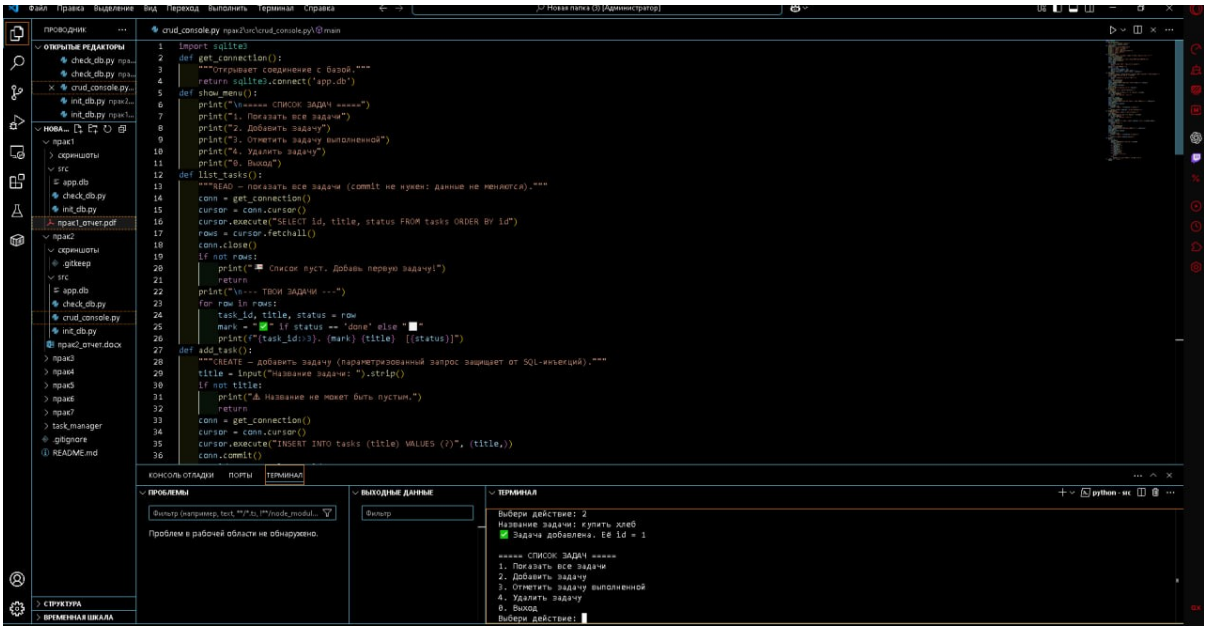


Рисунок 3 — добавление задачи «купить хлеб» (id = 1)

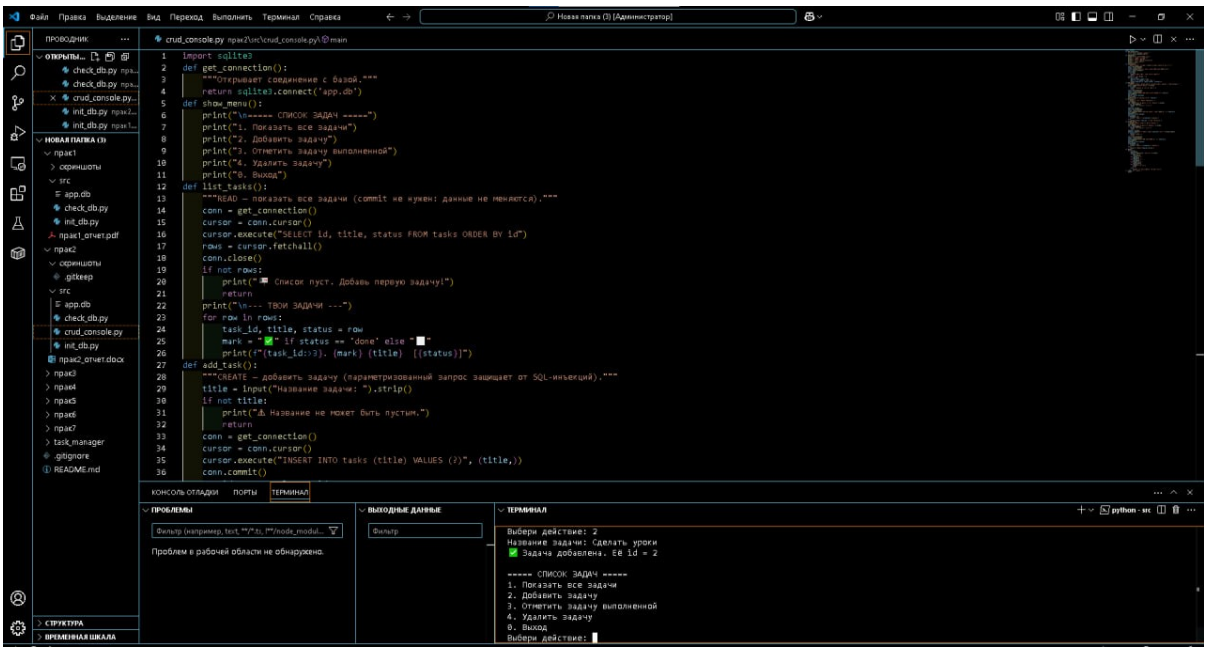


Рисунок 4 — добавление задачи «Сделать уроки» (id = 2)

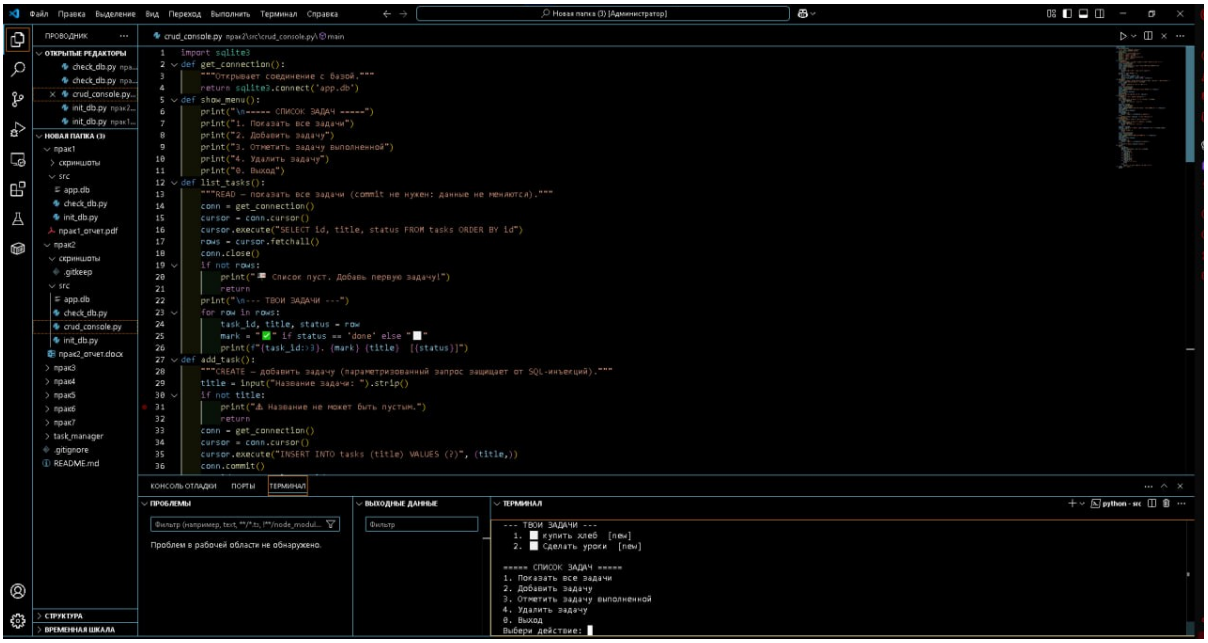


Рисунок 5 — список из двух задач со статусом new

Шаг 8. Сценарий проверки (продолжение)

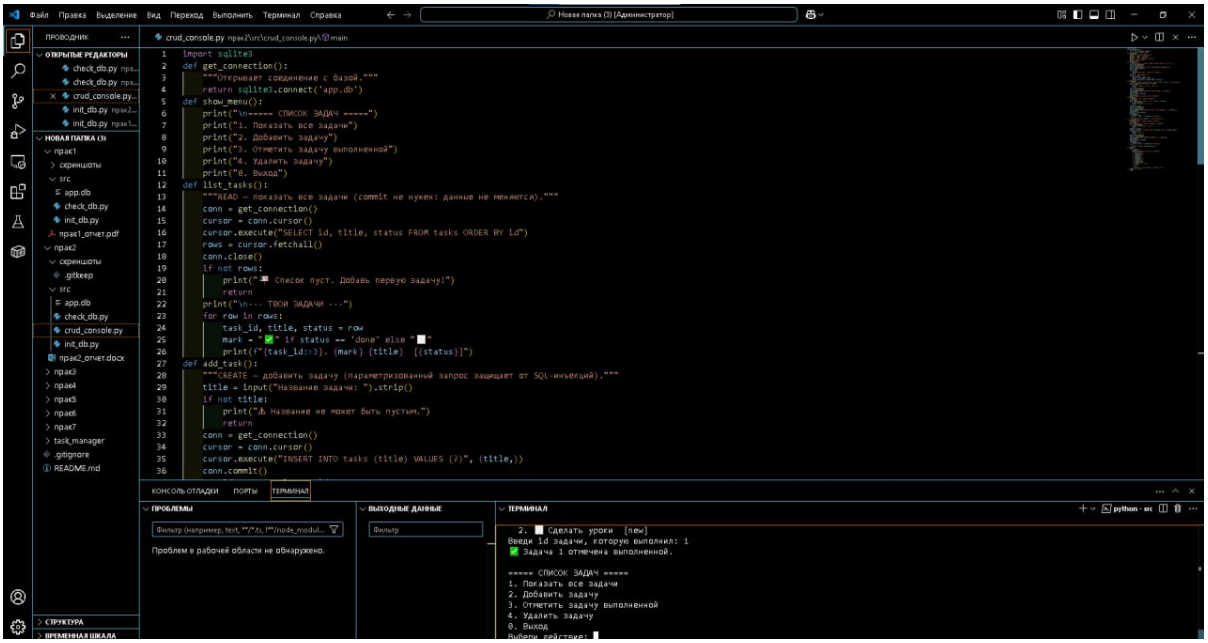


Рисунок 6 — задача 1 отмечена выполненной

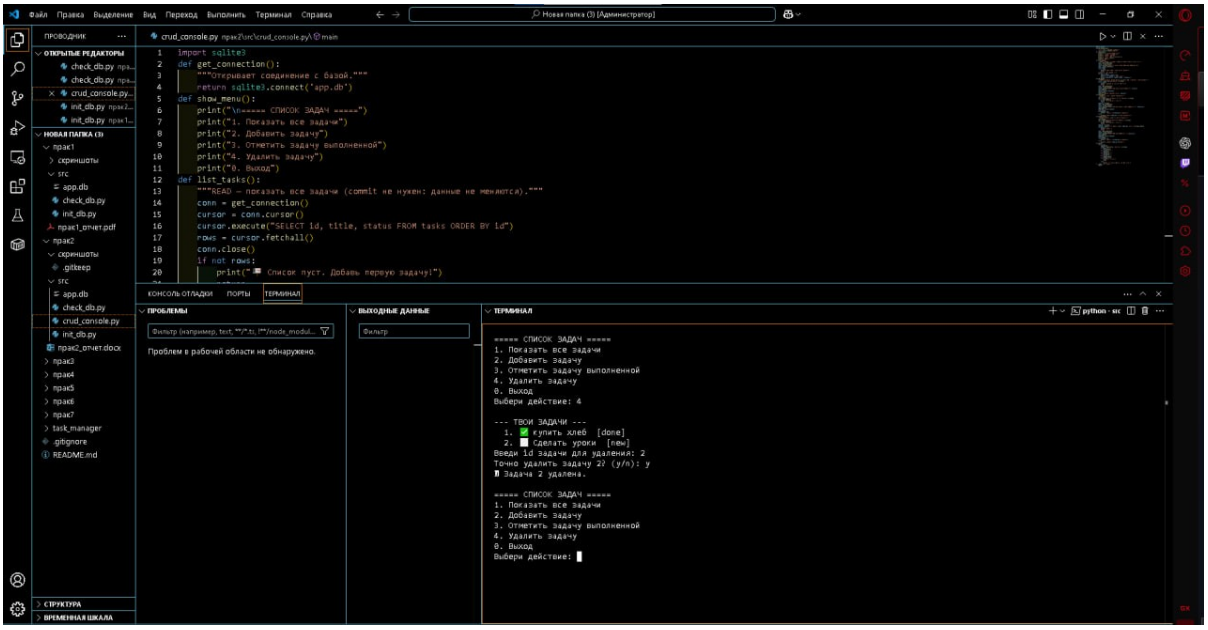


Рисунок 7 — удаление задачи 2 с подтверждением

Примеры запросов и ответов

```
Выбери действие: 2
Название задачи: Купить хлеб
✅ Задача добавлена. Её id = 1
Выбери действие: 3  → id: 1
✅ Задача 1 отмечена выполненной.
Выбери действие: 4  → id: 99  → y
⚠️ Задача с id=99 не найдена.
Название: '); DROP TABLE tasks; --   → сохранено как обычный текст
```

Ответы на контрольные вопросы

1. **Почему для SELECT не нужен commit(), а для INSERT — нужен?**

   SELECT только читает данные и ничего не меняет, фиксировать нечего. INSERT изменяет базу, и без commit() изменения не будут сохранены в файле.

2. **Что вернёт fetchone(), если в таблице нет строк?**

   None.

3. **Чем fetchone() отличается от fetchall()?**

   fetchone() возвращает одну строку (кортеж) или None — используется для запросов по одному id. fetchall() возвращает список всех строк — используется для вывода всего списка.

4. **Почему (title,) — это кортеж, а (title) — нет?**

   Кортеж из одного элемента в Python задаётся запятой. (title) — это просто значение в скобках, а (title,) — кортеж, который ожидает метод execute().

5. **Что вернёт cursor.rowcount после UPDATE, если такого id нет?**

   0 — запрос выполнился, но ни одна строка не изменилась.

6. **Зачем проверять isdigit() перед int(input())?**

   Чтобы при вводе нечислового значения (например, abc) программа не завершилась с ошибкой ValueError.

7. **Как параметризованные запросы защищают от SQL-инъекции?**

   Значение передаётся отдельно от текста SQL, и SQLite подставляет его как данные, а не как часть команды. Поэтому строка вида '); DROP TABLE tasks; -- сохраняется как обычный текст и не выполняется.

Вывод

Реализовано консольное приложение с полным набором CRUD-операций. Изучены параметризованные запросы, методы fetchone/fetchall, свойства lastrowid и rowcount, проверка ввода. Экспериментально подтверждена защита от SQL-инъекции.